

PESQUISA APROFUNDADA

Montagem de `/dev` mínimo em um runtime tipo Docker construído do zero em Python, Cython e C++

Feature analisada: runtime funcional sem `devpts` ainda, com `tmpfs`, `/dev/null`, `/dev/zero`, `/dev/random`, `/dev/urandom` e `/dev/tty`.

Contexto do projeto: control plane, metadados e API em Python; data plane, comandos e abstrações de baixo nível em C++; Cython como camada de integração e tradução.

Data: 12 de março de 2026.

Objetivo do documento: definir, contextualizar, analisar e propor um plano de implementação da feature sob as óticas de System Design, Low Level Design, Domain-Driven Design, padrões GoF, requisitos funcionais e não funcionais, segurança, testes e roadmap.

Sumário

1. Resumo executivo
2. Introdução e escopo da feature
3. Conceitos fundamentais: o que significa um `/dev` mínimo
4. Como Docker, OCI e runc tratam `/dev`
5. Contextualização no projeto em Python, Cython e C++
6. Proposta de System Design
7. Proposta de Low Level Design
8. Modelagem em Domain-Driven Design
9. Padrões de Design (GoF) aplicáveis
10. Requisitos funcionais
11. Requisitos não funcionais
12. Segurança, hardening e riscos de implementação
13. Plano de implementação por fases
14. Estratégia de testes e critérios de aceite
15. Conclusão
16. Anexos técnicos
17. Referências

1. Resumo executivo

A montagem de um `/dev` mínimo dentro de um container não é um detalhe cosmético; é parte da superfície de compatibilidade do Linux. A própria especificação OCI descreve que a ABI Linux não é composta apenas de syscalls, mas também de caminhos especiais de filesystem que as aplicações esperam encontrar, e por isso runtimes devem preparar arquivos especiais padrão, além de montar filesystems como `/proc`, `/dev/pts` e `/dev/shm` quando buscam conformidade completa [1]. O recorte deste documento, entretanto, é deliberadamente menor e mais pragmático: construir um primeiro marco funcional em que o runtime ainda não oferece `devpts`, mas já oferece um `/dev` isolado, efêmero, previsível e suficientemente compatível para cargas não interativas.

A decisão arquitetural central é simples e importante: o runtime não deve confiar no conteúdo de `/dev` vindo da imagem. Em vez disso, deve montar um novo `tmpfs` sobre o diretório `/dev` do rootfs do container e recriar apenas os nós explicitamente aprovados. Essa é a linha seguida por `runc/libcontainer` e refletida tanto na especificação OCI quanto no documento histórico de especificação do `runc` [1] [2]. Para o milestone aqui discutido, o perfil mínimo recomendado contém `/dev/null`, `/dev/zero`, `/dev/random`, `/dev/urandom` e `/dev/tty`, com seus respectivos majors, minors e permissões convencionais [4] [5] [6] [7]. O uso de `tmpfs` é tecnicamente apropriado porque fornece um sistema de arquivos temporário, cresce e encolhe sob demanda e perde conteúdo ao ser desmontado [17]. Ao mesmo tempo, esse `tmpfs` não deve ser montado com `nodev`, porque a própria finalidade do ponto de montagem é hospedar device nodes. Esse ponto parece trivial, mas é uma fonte recorrente de implementações erradas: `nodev` faz sentido em `/dev/shm`; não em `/dev`.

A limitação mais importante deste milestone é semântica, não apenas operacional. Criar `/dev/tty` não significa oferecer terminal interativo. O manual de `tty(4)` descreve `/dev/tty` como um sinônimo para o controlling terminal do processo, se ele existir [7]. Já a infraestrutura de pseudo-terminais depende de `devpts`, de `/dev/ptmx`, da criação de pares PTY master/slave e, em runtimes completos, do `bind mount` de um console para `/dev/console` quando o modo terminal está ativo [8] [3] [1] [2] [19]. Sem isso, o runtime pode rodar containers não interativos, batch jobs, processos em background e workloads com `stdio` ligado a pipes ou arquivos, mas não pode prometer o equivalente a `docker run -it`. Essa fronteira precisa aparecer tanto no design quanto no contrato da API.

Do ponto de vista do split tecnológico proposto, a recomendação mais forte deste documento é que Python permaneça dono do modelo de domínio, do metadata store, da API, da validação e do planejamento; C++ permaneça dono dos syscalls e da sequência de isolamento; e Cython seja tratado como uma camada de tradução tipada entre um plano imutável de execução e o executor de baixo nível. Em outras palavras, a fronteira Python/C++ não deve ser uma fronteira “de chamadas soltas”, mas uma fronteira de contrato. O controle em Python deve gerar um *Execution Plan* que inclua um *Mount Plan*, um *Device Plan*, um *Security Plan* e um *Process Plan*. O executor em C++ deve apenas materializar esse plano de forma determinística, com verificações fortes, tratamento explícito de

`errno`, uso de file descriptors para evitar TOCTOU e, sempre que possível, uso de APIs modernas como `openat2` para impedir resolução indevida de symlinks [24] [26].

Também há uma decisão de produto embutida. Se o projeto quer ser incremental e sólido, o melhor caminho é declarar este milestone como “perfil mínimo de `/dev` para containers não interativos”. Isso permite fechar compatibilidade básica para uma classe grande de workloads e, ao mesmo tempo, evita a armadilha de anunciar suporte a TTY quando a base ainda não suporta `devpts`. A própria evolução histórica do Docker mostra que o tratamento de PTY/console foi um passo específico, separado, com implicações reais para ferramentas como `tmux` e `screen` [20] [21]. No seu projeto, isso deve virar uma escolha consciente de roadmap, não um efeito colateral.

2. Introdução e escopo da feature

Quando se constrói um runtime de containers “do zero”, é tentador pensar em `/dev` como um pedaço secundário do filesystem, algo que pode ser deixado para depois. Essa intuição costuma gerar dois tipos de erro. O primeiro é de compatibilidade: processos aparentemente simples quebram porque assumem a existência de `/dev/null`, `/dev/urandom` ou `/dev/tty`. O segundo é de segurança: ao reaproveitar `/dev` da imagem, ou pior, ao bind-montar o `/dev` do host, o runtime abre uma superfície desnecessária de exposição de hardware, symlinks maliciosos e comportamentos difíceis de auditar. O modelo praticado por runtimes maduros é o oposto: `/dev` é reconstruído como parte do setup do rootfs e tratado como uma política explícita de virtualização de dispositivos [1] [2] [26].

O escopo deste documento é propositalmente preciso. A feature analisada é a montagem de um `/dev` mínimo, com runtime funcional sem `devpts` ainda. Em termos práticos, isso significa que o container deve ter um ponto de montagem `/dev` privado, baseado em `tmpfs`, e que esse ponto deve conter ao menos os nós `/dev/null`, `/dev/zero`, `/dev/random`, `/dev/urandom` e `/dev/tty` [5] [6] [7] [4]. O milestone não inclui ainda `/dev/pts`, `/dev/ptmx`, pseudo-TTY alocado pelo runtime, bind mount de console, resize de janela ou suporte interativo equivalente a `docker run -t` [8] [3] [19]. Essas capacidades aparecem neste documento como próximo estágio de evolução, não como parte do mesmo pacote.

Essa delimitação é importante porque a feature, embora pequena na superfície, atravessa quase todos os eixos estruturais de um runtime. Ela toca o System Design, porque exige um fluxo claro entre API, metadados, planejamento, execução privilegiada e observabilidade. Ela toca o Low Level Design, porque cada passo relevante é um syscall sensível: `mount`, `mknodat`, `pivot_root`, `chroot`, `openat2`, bind mounts, remounts e programação de cgroup de devices [10] [11] [12] [24] [15] [16]. Ela toca DDD porque introduz políticas de domínio reais: o que significa um “container não interativo”, quais dispositivos pertencem ao perfil mínimo, quando uma requisição deve ser rejeitada, e como rootful e user namespaces mudam a estratégia de provisionamento. Ela toca padrões de design porque a implementação pede estratégias, builders, estados, comandos e adaptadores de integração. E ela toca segurança de forma direta, porque `/dev/null` e

caminhos relacionados já apareceram em vulnerabilidades recentes do runc quando tratados sem endurecimento suficiente contra symlink attacks [23].

O objetivo, portanto, não é apenas dizer “crie estes cinco arquivos”. O objetivo é mostrar como essa feature deve ser tratada como um recorte completo de produto e arquitetura. O relatório assume Linux como plataforma alvo, privilegia referências primárias do kernel, da OCI, do Docker e do runc, e propõe um plano de implementação incremental compatível com a divisão Python/Cython/C++ solicitada. Onde o Docker completo oferece mais do que este milestone, o texto deixa clara a diferença. Onde o seu projeto pode aproveitar a experiência de runc, o texto extrai princípios que valem mesmo sem copiar a implementação em Go.

3. Conceitos fundamentais: o que significa um `/dev` mínimo

3.1 `/dev` como parte da ABI Linux do container

A especificação OCI é bastante explícita ao dizer que a ABI Linux inclui syscalls e também vários caminhos especiais de filesystem que as aplicações esperam encontrar [1]. Esse ponto muda a forma correta de enxergar o problema. Um container não é apenas um processo com namespaces e cgroups; ele é um ambiente de execução com um contrato visível ao user space. `/dev` é um pedaço central desse contrato porque várias bibliotecas, shells, utilitários e runtimes abrem device nodes por convenção. Em uma máquina “normal”, o diretório existe e é geralmente gerenciado por mecanismos do sistema como `udev`/`devtmpfs`. Em um container, esse comportamento não pode ser simplesmente herdado, porque a herança exporia dispositivos físicos e acoplaria o ambiente do container ao host. O que um runtime faz, então, é virtualizar a presença de dispositivos por meio de um filesystem novo e uma whitelist explícita.

Essa virtualização não precisa ser completa no primeiro dia, mas precisa ser coerente. Um `/dev` mínimo não é o mesmo que um `/dev` improvisado. O recorte mínimo coerente é aquele que atende expectativas básicas de compatibilidade, não vaza hardware indevido e mantém uma narrativa honesta sobre o que ainda não é suportado. Por isso, a presença de `/dev/tty` não deve ser usada como marketing de “TTY suportado”, e a ausência de `/dev/ptmx` deve ser tratada como uma limitação explícita do perfil.

3.2 `tmpfs`: por que montar `/dev` em memória

O manual de `tmpfs(5)` e a documentação do kernel descrevem `tmpfs` como um filesystem em memória virtual, temporário, cujo conteúdo desaparece quando a montagem é desmontada, e cujo uso de memória cresce e encolhe conforme a carga [17]. Para `/dev`, isso é quase ideal. Primeiro, porque device nodes não precisam persistir em disco entre execuções. Segundo, porque o estado de `/dev` deve ser reconstruído a cada container, não herdado cegamente da imagem. Terceiro, porque montar um filesystem novo por container cria uma fronteira clara de responsabilidade: o runtime passa a ser dono de tudo o que existe em `/dev`.

Há ainda uma razão negativa importante. Usar `devtmpfs` ou `bind-mountar` o `/dev` do host traria semântica demais. `devtmpfs` é um filesystem global gerenciado pelo kernel para expor dispositivos detectados; isso é excelente para o host, mas ruim para um container que deveria ver apenas um subconjunto controlado. Já o `bind mount` do `/dev` do host tende a vaziar dispositivos que a workload não precisa e complica bastante o enforcement pelo `cgroup` de devices. Portanto, o uso de `tmpfs` aqui não é um detalhe de performance; é uma decisão de isolamento.

Ao mesmo tempo, o `mount options` set deve ser pensado com cuidado. É razoável recomendar `nosuid`, `noexec`, `strictatime` e um `mode=0755` explícito para o diretório. Também é razoável aplicar um limite de tamanho moderado, não porque os device nodes consomem muito espaço, mas porque o runtime não deve deixar pontos de `tmpfs` sem governança. A documentação de `cgroup v2` mostra que dados de filesystem em cache, incluindo `tmpfs` e `shared memory`, entram na contabilidade de memória/`shmem` [16]. Em outras palavras, mesmo um `/dev` “pequeno” faz parte da governança de recursos do container. A nuance crítica é que esse `tmpfs` não deve receber a flag `nodev`; se receber, os device nodes criados ali deixam de cumprir sua função. O documento de especificação do `runc` ilustra isso ao montar `/dev` em `tmpfs` com `MS_NOEXEC` e `MS_STRICTATIME`, sem `MS_NODEV`, enquanto pontos como `/dev/shm` usam `MS_NODEV` [2].

3.3 `/dev/null`

Segundo `null(4)`, dados escritos em `/dev/null` são descartados, e leituras retornam `EOF`; o major/minor convencional é `1:3`, com modo tipicamente `0666` [5] [4]. Em termos de compatibilidade, é um dos dispositivos mais indispensáveis do sistema. Redirecionamentos de shell, wrappers de processos, scripts de inicialização, runtimes de linguagem e uma quantidade enorme de código legacy assumem sua existência. O `man page` é direto ao afirmar que, se `/dev/null` e `/dev/zero` não forem legíveis e graváveis por todos, muitos programas se comportarão de forma estranha [5]. Portanto, para o perfil mínimo, não há espaço para experimentação: o runtime deve criar exatamente um char device com o major/minor correto, modo compatível e verificação posterior do inode criado.

Há também um componente de segurança que subiu muito de importância. O changelog do `runc` de 2025 registrou correções para vulnerabilidades em que o inode de `/dev/null` dentro do container era usado de forma insegura durante a implementação de `masked paths`; se um atacante conseguisse substituir `/dev/null` por um `symlink`, o runtime poderia `bind-mountar` o alvo do link em vez do dispositivo esperado [23]. A lição para um runtime novo é clara: `/dev/null` não pode ser tratado como uma “coisa trivial” que você cria e depois esquece. Ele é um objeto de alta confiança. O caminho até ele deve ser endurecido contra `symlink` traversal, o inode final deve ser verificado, e o runtime deve assumir que o `rootfs` pode ser malicioso.

3.4 `/dev/zero`

`/dev/zero` é o char device `1:5` que descarta escritas e retorna bytes zero em leituras [5] [4]. Embora pareça menos “essencial” do que `/dev/null`, ele continua relevante para compatibilidade real. Ferramentas como `dd`, testes de throughput, geradores simples de

payload, rotinas antigas de inicialização de arquivos e alguns caminhos de fallback de bibliotecas ainda o usam. Há, de novo, pouca vantagem em tentar ser criativo. Criá-lo corretamente custa quase nada, e omiti-lo só desloca complexidade para o usuário.

3.5 `/dev/random` e `/dev/urandom`

O manual de `random(4)` define `/dev/random` e `/dev/urandom` como interfaces para o gerador de números aleatórios do kernel, com major/minor `1:8` e `1:9`, respectivamente [6] [4]. O mesmo manual observa que Linux fornece `getrandom(2)` desde 3.17 como interface mais simples e segura, sem depender de arquivos especiais [6] [25]. Em implementações modernas, isso pode induzir a pergunta errada: “já que existe `getrandom`, eu posso omitir esses device nodes?” Do ponto de vista de compatibilidade de runtime, a resposta prática é não. Muitas aplicações atuais já usam `getrandom`, mas muitas outras continuam abrindo `/dev/urandom` diretamente, seja por legado, seja por portabilidade, seja por hábitos do ecossistema.

O relatório recomenda fortemente que o runtime não tente “emular” aleatoriedade no user space. A única decisão correta é expor os device nodes reais do kernel, com majors/minors corretos, e deixar a política criptográfica com o próprio kernel. `random(4)` ainda descreve `/dev/random` como interface legada que bloqueia quando necessário, enquanto `/dev/urandom` oferece bytes pseudoaleatórios sem bloqueio em leituras normais, salvo nuances de early boot [6]. Em um container, você não quer reinterpretar essa semântica. Você quer apenas torná-la acessível de forma segura e compatível.

3.6 `/dev/tty`

`tty(4)` é provavelmente o item mais fácil de interpretar errado. O manual diz que `/dev/tty` é um char device `5:0`, usualmente modo `0666`, que atua como sinônimo para o controlling terminal do processo, se ele existir [7] [4]. A expressão “se ele existir” é o coração do problema. O node em si não cria um terminal; ele apenas fornece um ponto de acesso ao terminal de controle já associado ao processo. Portanto, em containers não interativos lançados com stdio em pipes, `/dev/tty` pode existir e, ainda assim, a abertura falhar, porque o processo não tem controlling terminal. Isso é um comportamento correto, não um bug.

A implicação prática é dupla. De um lado, ainda vale a pena incluir `/dev/tty` no perfil mínimo, porque muitas aplicações usam a tentativa de abrir `/dev/tty` como forma de detectar se estão ligadas a um terminal e de ajustar seu comportamento. De outro lado, a API do seu runtime não pode sugerir que “`/dev/tty` presente” equivale a “sessão TTY suportada”. Essas são afirmações diferentes. No milestone atual, o contrato deve ser: existe um node compatível com a ABI Linux; o runtime ainda não oferece infraestrutura PTY própria.

3.7 O que `devpts` adicionaria e por que ele ainda está fora do escopo

`pts(4)` e a documentação do kernel para `devpts` descrevem a base dos pseudo-terminais modernos no Linux: abrir `/dev/ptmx` aloca um PTY master, cria um slave correspondente em `/dev/pts` e permite que programas obtenham o path do slave com funções como `ptsname`, `grantpt` e `unlockpt` [8] [3]. A documentação do kernel também

destaca que cada mount de `devpts` é distinto, que `/dev/pts/ptmx` nasce com modo `0000` e que, se a estratégia for usar symlink ou bind mount, o mount deve considerar `ptmxmode=0666` [3]. Essa complexidade simplesmente não está presente no milestone atual.

Isso não é um problema; é uma escolha de engenharia. `devpts` é a fronteira natural entre “container não interativo compatível” e “container com experiência de terminal semelhante ao Docker”. Ao empurrá-lo para o milestone seguinte, o projeto ganha a chance de fechar primeiro o caminho de mounts, rootfs, device policy, cgroup de devices, hardening de paths e integração Python/C++ sem o peso adicional de pseudo-TTY. Desde que a limitação seja explícita, isso é uma decisão correta. O erro seria misturar as duas coisas e lançar um suporte “meio TTY” que passa em alguns demos, mas falha em cenários reais como `tmux`, `screen`, programas que exigem echo-off ou comandos que abrem `/dev/ptmx`.

4. Como Docker, OCI e runc tratam `/dev`

4.1 O recorte histórico: do Docker 0.9 à pilha moderna

Historicamente, o Docker não nasceu com a pilha atual. O anúncio do Docker 0.9, em março de 2014, marcou a introdução de `libcontainer` como driver nativo em Go para acessar diretamente namespaces, cgroups, capabilities, perfis AppArmor e outros mecanismos do kernel, reduzindo dependências como LXC [20]. Esse ponto interessa ao seu projeto porque ele mostra um padrão recorrente em runtimes maduros: o coração da execução de containers tende a migrar para uma camada especializada e mais próxima do kernel, enquanto a experiência de produto e orquestração permanece em uma camada superior. O split que você propôs — Python para control plane e C++ para data plane — segue exatamente essa lógica, embora com linguagens diferentes.

Na pilha atual, a documentação do Docker registra que o Docker Engine usa `containerd` para gerenciamento do ciclo de vida dos containers e, por padrão, o `containerd` usa o `runc` como runtime [18]. Isso importa porque, quando se fala em “como o Docker trata `/dev`”, o comportamento prático de referência hoje é, em boa medida, o comportamento implementado pelo `runc/libcontainer` sob o contrato OCI. Em outras palavras, não é necessário “imitar o Docker daemon monolítico antigo”; o alvo mais útil é entender o pipeline que a pilha atual consolidou.

4.2 O modelo OCI

A especificação OCI diz que runtimes devem considerar default filesystems como `/proc`, `/sys`, `/dev/pts` e `/dev/shm`, e que, além de quaisquer devices configurados explicitamente, o runtime deve fornecer por padrão `/dev/null`, `/dev/zero`, `/dev/full`, `/dev/random`, `/dev/urandom`, `/dev/tty`, mais `/dev/console` quando terminal está habilitado, e `/dev/ptmx` como bind mount ou symlink de `/dev/pts/ptmx` [1]. Esse texto é útil por dois motivos. Primeiro, ele define um alvo de compatibilidade completa. Segundo, ele deixa evidente que o milestone deste documento é um subconjunto intencional dessa conformidade.

Em termos de produto, isso leva a uma recomendação simples: o seu runtime deve documentar que o “perfil mínimo de `/dev`” não é o perfil OCI completo. Ele entrega o essencial para containers não interativos, mas ainda não implementa terminal completo, `/dev/full` e demais componentes auxiliares. Essa honestidade de contrato simplifica o debugging e evita uma classe de tickets confusos em que o usuário assume paridade total com runc.

4.3 O modelo runc/libcontainer

O documento `libcontainer/SPEC.md` é particularmente valioso porque descreve de forma operacional o que o runtime faz ao montar o filesystem do container: `/proc` em `proc`, `/dev` em `tmpfs`, `/dev/shm` em outro `tmpfs`, `/dev/mqueue` em `mqueue`, `/dev/pts` em `devpts` e `/sys` em `sysfs` [2]. O mesmo documento explica que, depois de montar esses filesystems, o runtime popula `/dev` com device nodes padrão, cria symlinks como `/dev/fd`, `/dev/stdin`, `/dev/stdout` e `/dev/stderr` após `/proc`, e trata `/dev/console` quando um pseudo-TTY é fornecido [2].

Ainda mais revelador é o código atual do runc em `rootfs_linux.go`. Ali aparecem várias decisões de engenharia que valem como princípios de projeto. O runc cria symlinks auxiliares em `/dev`, reabre descritores apontando para `/dev/null` após entrar no rootfs do container quando necessário, escolhe entre `mknod` e `bind mount` para devices dependendo de user namespace, ignora `/dev/ptmx` no loop genérico porque trata esse caminho com setup específico, verifica inode type e major/minor depois do `mknodat`, e torna a propagação do mount privada/slave para evitar vazamento de mounts ao host [26]. Esse é exatamente o tipo de material que se transforma em requisitos internos para o seu runtime.

4.4 O que o comportamento do Docker ensina sobre o seu milestone

Há uma nota histórica útil nos release notes antigos do Docker: a mudança que “mount-bind the PTY as container console” foi citada como habilitando `tmux` e `screen` [21]. Isso é tecnicamente interessante porque confirma que terminal funcional em containers não é apenas “passar stdin/out”. Há uma infraestrutura concreta por trás. O seu projeto, portanto, deve tratar “`/dev` mínimo sem `devpts`” como o estágio imediatamente anterior a esse mundo. Se você fechar bem esse estágio, o passo seguinte para PTY fica muito mais nítido.

Há também um aprendizado de segurança. O runc precisou corrigir, em 2025, vulnerabilidades relacionadas ao uso de `/dev/null` e `/dev/console` em `bind mounts` e `masked paths` [23]. O recado não é “copie o bug”; o recado é “trate qualquer caminho sob `/dev` como superfície sensível”. Um runtime novo tem vantagem aqui: ele pode incorporar o endurecimento desde o início, sem o peso de compatibilidade retroativa.

5. Contextualização no projeto em Python, Cython e C++

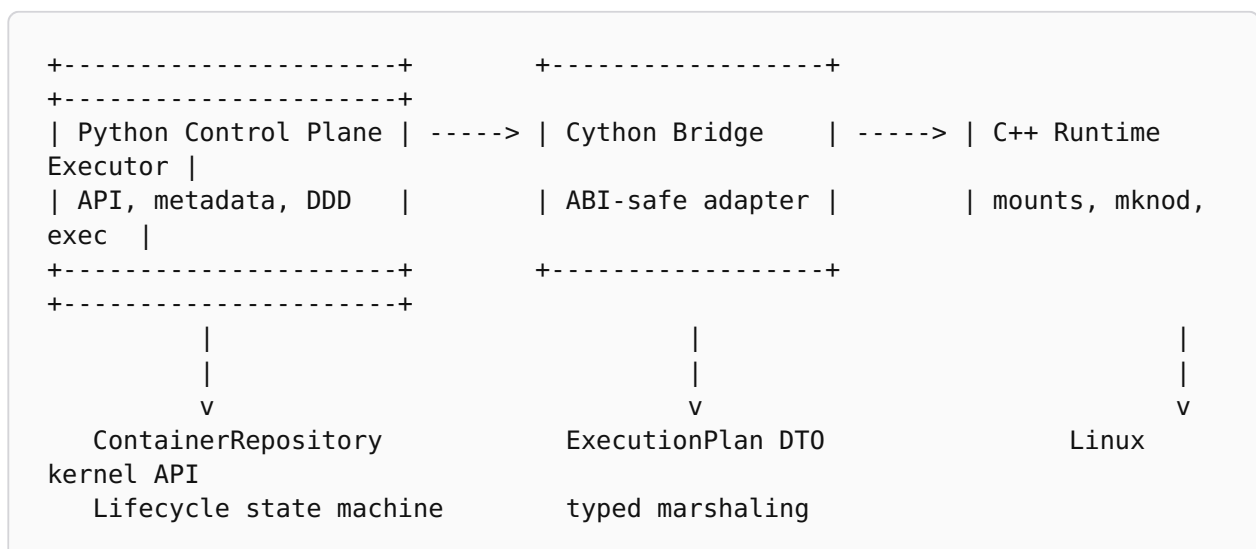
O split proposto — Python para control plane, metadados e API; C++ para data plane e abstrações próximas do hardware/kernel; Cython como cola — é sólido, desde que a separação seja desenhada como separação de responsabilidade e não apenas como

separação de linguagem. O erro clássico seria deixar Python disparar pequenas chamadas imperativas para C++ do tipo “agora monte isso”, “agora crie aquilo”, “agora faça pivot”, criando uma coreografia distribuída, difícil de reproduzir e difícil de auditar. A recomendação aqui é diferente: Python deve planejar; C++ deve executar; Cython deve traduzir um contrato estável entre os dois lados.

Isso significa que a unidade de integração não deveria ser o syscall, mas o plano. Quando a API de alto nível recebe uma requisição para criar um container, o domínio em Python deve produzir um objeto imutável que representa a intenção completa daquela execução. Esse objeto pode ser chamado de *ExecutionPlan* e conter ao menos um *NamespacePlan*, um *RootfsPlan*, um *MountPlan*, um *DevicePlan*, um *CgroupPlan*, um *CapabilityPlan* e um *ProcessLaunchPlan*. Cython traduz esse grafo de value objects para uma representação ABI-safe consumível pelo executor em C++. A vantagem é imediata: toda a política de negócio continua em Python, onde é mais fácil evoluir regras, persistir metadados, validar contratos e produzir erros semânticos. Já a parte sensível ao kernel fica concentrada em um executor determinístico.

No contexto específico da feature de `/dev` mínimo, isso é especialmente útil porque o control plane precisa decidir coisas que não são apenas técnicas. Ele precisa saber se o usuário pediu terminal interativo; precisa verificar se o runtime está em modo rootful ou user namespace/rootless; precisa decidir se a ausência de `devpts` vira rejeição imediata ou um downgrade explícito; precisa persistir no metadata store qual perfil de devices foi aplicado; e precisa expor essa informação de volta por API e observabilidade. Tudo isso é problema de domínio, não de syscall.

Já o executor em C++ deve ser visto como um componente de confiança elevada e escopo estreito. Seu trabalho é: criar namespace(s), preparar mounts, criar device nodes ou bind-mountá-los, programar o cgroup de devices, trocar o root, ajustar capabilities, lançar o processo e devolver resultados estruturados. Essa sequência é naturalmente mais segura se implementada em um processo de vida curta, separado do daemon Python de longa duração. Mesmo que o projeto use Cython como bridge principal, a recomendação prática é que o código privilegiado em C++ possa rodar como helper isolado ou subprocesso executor. Isso reduz blast radius e melhora a auditabilidade do caminho privilegiado sem abrir mão do split tecnológico desejado.



namespaces, cgroups, Validation & policy mknodat, pidfd	error translation	mount,
---	-------------------	--------

A escolha de Cython também ganha um papel conceitual importante: ela vira uma *anti-corruption layer* entre o domínio do control plane e o domínio dos detalhes do kernel. Isso ajuda a preservar um vocabulário estável no lado Python — “perfil mínimo de /dev”, “modo terminal desabilitado”, “estratégia de provisionamento por bind mount”, “container em estado creating/failed/running” — sem contaminar o modelo com detalhes acidentais como bits de mount flags e números mágicos de major/minor espalhados por todo o código.

6. Proposta de System Design

6.1 A ideia central: um pipeline determinístico de preparação do rootfs

No nível de System Design, a feature deve ser tratada como um pipeline de preparação do ambiente do container. O pedido chega à API do control plane, é validado contra capacidades suportadas, gera um plano imutável, é persistido como intenção de criação, e então é materializado por um executor especializado. O design mais saudável não é o de um daemon Python que vai “tomando pequenas decisões” à medida que syscalls falham; o design saudável é o de um planner que resolve ambiguidade e um executor que materializa uma decisão já tomada. Isso separa erro de política de erro de infraestrutura.

O fluxo recomendado é o seguinte. A API recebe uma requisição de criação contendo imagem/rootfs, comando, namespaces desejados, limites, política de devices e um campo explícito para terminal. O domínio em Python resolve a política do pedido. Se `terminal=true` ou equivalente foi solicitado, e o runtime ainda está no milestone sem `devpts`, a decisão correta é rejeitar a requisição antes de qualquer tentativa de execução, com erro semântico claro. Se o pedido é compatível com o perfil não interativo, o planner monta um *MinimalDevProfile* e o incorpora ao *ExecutionPlan*. Esse plano, uma vez persistido, é entregue ao executor C++ via Cython.

A criação do container passa então por estados explícitos. Uma sequência suficientemente robusta seria: *Planned*, *Creating*, *RootfsPrepared*, *Isolated*, *Running*, *Exited*, *Failed* e *Destroyed*. O ponto importante não é o nome exato dos estados, mas o fato de que a montagem de /dev mínimo deixa de ser um side effect implícito e passa a ser um passo identificável no lifecycle. Isso melhora muito o debugging: o usuário consegue saber se o container falhou antes do pivot, durante a criação dos devices ou já dentro do novo root.

6.2 Componentes lógicos do sistema

O sistema pode ser visto em cinco componentes lógicos. O primeiro é o *API and Metadata Service* em Python, responsável por expor endpoints, autenticação, serialização, persistência, listagem e inspeção. O segundo é o *Planning Service*, também em Python, que transforma uma intenção de container em um plano executável e resolve políticas de

compatibilidade. O terceiro é a *Bridge Layer* em Cython, que valida tipos, traduz estruturas e converte códigos de erro e exceções. O quarto é o *Runtime Executor* em C++, dono dos syscalls e do isolamento. O quinto é o *Telemetry and Audit Layer*, que registra eventos, logs estruturados, mount topology e resultados de verificação.

Para a feature específica, o *Planning Service* precisa produzir pelo menos quatro subplanos que devem permanecer coerentes entre si. O *MountPlan* define que `/dev` será um `tmpfs` novo, montado no mount namespace do container. O *DevicePlan* define quais nodes existirão e qual estratégia será usada para provisioná-los: `mknod` em modo `rootful` ou `bind mount` de dispositivos confiáveis do host em modo `user namespace/rootless`. O *CgroupPlan* define as regras de devices correspondentes à mesma whitelist. E o *SecurityPlan* define quais capabilities e operações serão permitidas até o fim do setup e quais serão removidas antes do `execve`. O maior erro de design aqui seria gerar esses quatro planos de forma independente; eles precisam nascer da mesma política para não divergirem.

6.3 Sequência de execução recomendada

```
CreateContainer(request)
-> validate feature flags and terminal policy
-> persist ContainerAggregate(status=Planned)
-> build ExecutionPlan
-> spawn C++ executor
    -> create child with namespaces
    -> set mount propagation to private/slave
    -> bind mount rootfs onto itself if needed
    -> mount tmpfs on rootfs/dev
    -> create or bind approved device nodes
    -> program devices cgroup
    -> pivot_root or fallback strategy
    -> drop setup capabilities
    -> exec init process
-> return pid/pidfd + state transition
```

Essa sequência pode ser refinada com recursos modernos. Em kernels recentes, o uso de `clone3` com retorno de `pidfd` é vantajoso porque fornece um identificador de processo estável e evita corridas associadas à reciclagem de PIDs; o ecossistema de `pidfd` também permite sinalização e observação mais seguras [27] [28]. Isso não é estritamente necessário para o `/dev` mínimo, mas é uma decisão arquitetural coerente para um runtime novo, especialmente quando o control plane precisa monitorar containers de forma confiável.

6.4 Rootful e rootless como estratégias de produto, não apenas como detalhe técnico

O design do sistema deve modelar desde já a diferença entre `rootful` e `user namespace/rootless`. A documentação do kernel e do Docker é clara ao registrar que, embora processos em `user namespace` possam ganhar várias capacidades no namespace, a

criação de device nodes via `mknod` continua sendo restrita ao initial user namespace; o próprio Docker documenta a impossibilidade de usar `mknod` em containers user-namespaced [14] [22] [13]. Isso significa que a estratégia de provisionamento de devices não é um detalhe de implementação escondido. É parte da capacidade operacional do runtime.

A recomendação deste relatório é representar isso no domínio como uma estratégia explícita. Em modo rootful, o executor pode criar os devices diretamente com `mknodat`. Em modo user namespace/rootless, o executor deve fazer bind mount de dispositivos confiáveis do host, exatamente como o runc faz ao detectar esse cenário [26]. Se o produto ainda não quiser suportar rootless neste milestone, a API deve dizê-lo claramente; mas a arquitetura deve deixar o ponto de extensão preparado. O que não vale é fingir que o mesmo caminho serve para ambos.

6.5 Disponibilidade, rollback e idempotência

Embora este seja um runtime local, a feature também tem implicações de operabilidade. Se a montagem de `/dev` falhar no meio do setup, o sistema precisa ter rollback determinístico. No mount namespace do child, isso é mais simples porque o kernel desmonta o conjunto ao destruir o namespace, mas o control plane ainda precisa registrar o fracasso, coletar telemetria suficiente e evitar estados fantasmas na base de metadados. Por isso, a recomendação é registrar cada passo material do setup como evento auditável: *DevTmpfsMounted*, *DeviceProvisioned*, *DevicesCgroupApplied*, *RootPivoted*, *InitExecSucceeded* e assim por diante.

A idempotência também merece tratamento explícito. Se uma tentativa de criação falha após persistir o plano, uma segunda tentativa de criação com o mesmo identificador não deve reaproveitar um rootfs parcialmente modificado ou mounts residuais no host. Montar um `tmpfs` novo sobre `/dev` por tentativa ajuda bastante nesse ponto, porque o estado é naturalmente descartável. Ainda assim, diretórios auxiliares, cgroups e logs temporários devem ter cleanup previsível.

7. Proposta de Low Level Design

7.1 Módulos e responsabilidades no executor C++

No C++, vale a pena decompor o executor em módulos pequenos, com responsabilidades inequívocas. Um módulo *KernelFeatureProbe* detecta, durante a inicialização do runtime, a presença de APIs relevantes como `openat2`, `clone3`, `pidfd` e backend de devices cgroup disponível. Um módulo *PathResolver* encapsula toda a lógica de abertura segura de caminhos relativos ao rootfs usando directory file descriptors, `O_NOFOLLOW` e, quando disponível, `openat2` com flags de resolução restritiva [24]. Um módulo *MountManager* monta filesystems, ajusta propagação e gerencia remounts. Um módulo *DeviceProvisioner* sabe materializar um *DeviceSpec* por `mknod` ou bind mount. Um módulo *CgroupDeviceManager* traduz o *DevicePlan* para regras de cgroup v1 ou v2. Um módulo *RootSwitcher* encapsula `pivot_root` ou fallback. E um módulo *ProcessLauncher* faz `execve` e devolve pid/pidfd ao control plane.

Essa decomposição tem dois ganhos. O primeiro é testabilidade: torna viável injetar um *SyscallFacade* falso e validar a sequência de chamadas sem criar namespaces de verdade. O segundo é clareza operacional: quando o runtime falhar, será mais fácil atribuir a falha a “resolução de path”, “montagem de tmpfs”, “provisionamento de device”, “cgroup programming” ou “root switch”. Em runtime, erros sem contexto são quase sempre mais caros do que erros puros.

7.2 Estruturas de dados mínimas

Um *DeviceSpec* deveria carregar, no mínimo, os campos `path`, `type`, `major`, `minor`, `mode`, `uid`, `gid` e `provisioningMethod`. Um *MountSpec* deveria carregar `source`, `target`, `fstype`, `flags` e `data/options`. Um *ExecutionPlan* deveria conter listas ordenadas de mounts e devices, além de campos como `rootfs`, `argv`, `env`, `namespaces`, `capabilities`, `cwd` e modo terminal. É recomendável que a ordenação seja parte do contrato. Montar `/dev` antes de criar os nodes, por exemplo, não é uma preferência de implementação; é uma dependência lógica.

No lado do Cython, o ideal é traduzir o plano de Python para estruturas simples e estáveis, sem objetos complexos com ownership ambíguo. Para um runtime desse tipo, a melhor cola costuma ser uma combinação de POD-like structs, vetores e enums explícitos. Isso reduz o risco de vazamento de memória e torna mais simples a evolução do ABI.

7.3 Algoritmo recomendado para a montagem de `/dev` mínimo

A sequência de baixo nível recomendada começa antes do `exec`. O processo executor deve entrar no mount namespace do container ou criar um novo com `CLONE_NEWNS`, então ajustar a propagação do root para `MS_PRIVATE|MS_REC` ou `MS_SLAVE|MS_REC`, evitando que mounts futuros escapem para outros namespaces [9] [26] [11]. Em seguida, deve garantir que o `rootfs` seja um mount point adequado para a troca de root, eventualmente via bind mount sobre si mesmo, e abrir o `rootfs` por file descriptor.

Depois disso vem a parte central da feature. O runtime deve resolver com segurança o caminho relativo `dev` dentro do `rootfs`. Se o path não existir, deve criar um diretório. Se existir como diretório, deve reutilizar o ponto de montagem. Se existir como symlink ou outro tipo incompatível, a recomendação mais segura é rejeitar o `rootfs` ou sanitizá-lo por meio de uma política explícita; o importante é nunca seguir cegamente esse caminho. Com o diretório seguro em mãos, o runtime monta `tmpfs` em `<rootfs>/dev`, usando opções equivalentes a `nosuid,noexec,strictatime,mode=755` e eventualmente um `size=` pequeno e previsível. O próprio ato de montar um novo `tmpfs` tem valor de segurança porque ele neutraliza conteúdo pré-existente do `/dev` da imagem.

Com o `tmpfs` montado, o runtime percorre o *DevicePlan*. Para o milestone atual, o conjunto recomendado é:

Path	Tipo	Major:Minor	Modo	Justificativa
<code>/dev/null</code>	char	1:3	0666	Descartar escritas e retornar EOF em leituras; compatibilidade universal.

/dev/zero	char	1:5	0666	Fornecer bytes zero; muito usado por utilitários e testes.
/dev/random	char	1:8	0666	Interface legada do RNG do kernel.
/dev/urandom	char	1:9	0666	Interface amplamente usada para entropia em user space.
/dev/tty	char	5:0	0666	Alias para controlling terminal do processo, se houver.

Em modo rootful, cada item deve ser criado com `mknodat`. Em modo user namespace/rootless, o runtime deve fazer bind mount de um device node confiável do host para o caminho correspondente no rootfs do container, tal como o runc faz ao detectar user namespace [26] [14] [22]. Em ambos os casos, a regra crítica é a mesma: depois da criação ou do bind mount, o runtime deve reabrir o inode final com `O_PATH/O_NOFOLLOW` e verificar que ele continua sendo o tipo esperado e que o `rdev` corresponde ao major/minor planejado. Essa verificação, presente no código do runc, vale ouro para defesa contra TOCTOU [26].

Depois dos device nodes, o runtime pode — mas só se `/proc` já estiver ou vier a ser montado — criar symlinks auxiliares como `/dev/fd`, `/dev/stdin`, `/dev/stdout` e `/dev/stderr`, que apontam para `/proc/self/fd` [2] [26]. Esses symlinks melhoram compatibilidade, mas não devem ser criados de forma cega se `/proc` ainda não fizer parte do ambiente do container.

7.4 O que não deve existir neste milestone

É tecnicamente melhor que `/dev/ptmx`, `/dev/pts` e `/dev/console` simplesmente não existam no perfil mínimo, ou existam apenas quando já houver infraestrutura correta para eles. Criar placeholders quebrados é pior do que não criar. Muitas bibliotecas usam a presença de certos arquivos especiais como sinal de capacidade. Se o runtime expõe `/dev/ptmx` mas não montou `devpts` de maneira correta, a aplicação falha mais tarde e de forma menos compreensível. Esse tipo de “quase suporte” custa muito em manutenção.

O mesmo vale para permissões e ownership rebuscados. Em distribuições tradicionais, `/dev/tty` costuma ser `root:tty` com modo `0666` [7]. Em um primeiro milestone, especialmente se o runtime ainda não tiver gestão consistente de grupos internos ao namespace, usar `root:root` com `0666` é aceitável para esses cinco nodes, desde que a política seja documentada. O detalhe material é o modo, o tipo do inode e o major/minor; o refinamento de ownership pode evoluir depois.

7.5 Integração com cgroups de devices

Criar o inode não basta. O controlador de devices do cgroup é quem define se processos do container podem de fato criar ou abrir device files. A documentação do cgroup v1 descreve um whitelist model com tipo, major, minor e bits de acesso `r`, `w` e `m` [15]. Em cgroup v2, o controle é implementado sobre BPF e continua governando tanto acesso a

devices existentes quanto criação de novos device nodes [16]. Daí nasce uma regra de design muito importante: o *DevicePlan* e o *CgroupPlan* devem ser gerados a partir do mesmo conjunto de *DeviceSpec*.

Para o perfil mínimo, isso significa permitir ao menos: `c 1:3 rwm`, `c 1:5 rwm`, `c 1:8 rwm`, `c 1:9 rwm` e `c 5:0 rwm`. A política recomendada é negar por padrão o restante. Sem essa coerência, o runtime cai em uma das duas metades erradas: ou expõe um inode que o processo não pode usar, gerando erros confusos de permissão, ou libera access rules para devices que nem sequer deveriam estar visíveis.

7.6 Troca do root e sequência final

Depois de montar `/dev` e preparar o resto do rootfs, o runtime precisa prender o processo ao novo filesystem. `pivot_root(2)` é o caminho clássico para isso, desde que o rootfs seja um mount point adequado e que a propagação do mount esteja ajustada corretamente [11] [2] [26]. O documento histórico do runc também menciona o fallback com `MS_MOVE` + `chroot` para cenários em que `pivot_root` não é suportado, como rootfs em `ramfs` [2]. Em um runtime novo, essa lógica pode ser encapsulada em um *RootSwitcher* com duas estratégias, escolhidas por capability probing e pelo tipo de rootfs.

Após a troca de root, o executor deve considerar reabrir descritores que apontavam para `/dev/null` no rootfs anterior, como o runc faz em seu helper `reOpenDevNull`, para garantir que o `/dev/null` usado por stdin/out/err seja o do container e não o do contexto anterior [26]. Em seguida, deve reduzir capabilities, configurar `no_new_privs` se essa política fizer parte do produto, aplicar `seccomp` se houver, e só então chamar `execve`.

7.7 Esboço de pseudoimplementação

```
int RuntimeExecutor::launch(const ExecutionPlan& plan) {
    probeKernelFeatures();
    int pidfd = -1;
    pid_t child = clone3_with_pidfd(plan.namespaces(), &pidfd);

    if (child == 0) {
        make_root_mount_private_or_slave();
        prepare_rootfs_mountpoint(plan.rootfs());
        safe_mount_tmpfs_dev(plan.rootfs(),
            "nosuid,noexec,strictatime,mode=755,size=64k");

        for (const DeviceSpec& dev : plan.device_plan().devices()) {
            if (dev.provisioningMethod == ProvisioningMethod::Mknod) {
                secure_mknodat_verified(plan.rootfs_fd(), dev);
            } else {
                secure_bind_mount_device(plan.rootfs_fd(), dev);
            }
        }

        apply_devices_cgroup(plan.cgroup_plan());
        switch_root(plan.rootfs());
        maybe_reopen_dev_null();
    }
}
```

```
        drop_setup_capabilities(plan.capability_plan());
        execve(plan.argv()[0], plan.argv(), plan.envp());
        _exit(errno_to_exit_code(errno));
    }

    return pidfd;
}
```

A função acima não é código pronto, mas resume a ideia correta: o executor materializa um plano, não “descobre o que fazer” em tempo de erro. Esse é o tipo de linha-mestra que evita uma arquitetura frágil.

8. Modelagem em Domain-Driven Design

8.1 Por que DDD faz sentido num runtime

É comum subestimar DDD em projetos de runtime porque o tema parece “baixo nível demais”. Na prática, um runtime de containers tem muito domínio. Ele precisa representar estados, políticas, perfis de compatibilidade, diferenças entre rootful e rootless, invariantes de segurança e contratos de API. Sem um modelo de domínio explícito, essas regras acabam espalhadas entre ifs em Python, flags em Cython, enums em C++ e mensagens de erro improvisadas. O resultado é um sistema funcional, porém sem linguagem unificada. A feature de `/dev` mínimo é um bom exemplo de como o domínio emerge naturalmente.

Do ponto de vista de DDD, o conceito central aqui é o de um *ContainerAggregate*. Esse agregado representa a intenção, o estado e as políticas aplicadas a uma instância de container. Ele não executa syscalls; ele guarda decisões de domínio. A montagem de `/dev` mínimo seria então uma política pertencente ao agregado ou a um serviço de domínio associado, não uma função solta em alguma camada de infraestrutura.

8.2 Bounded contexts sugeridos

Um desenho saudável separa pelo menos quatro bounded contexts. O primeiro é o contexto de *Container Lifecycle*, que cuida de criação, start, stop, kill, inspect e estados. O segundo é o contexto de *Filesystem & Mount Topology*, que modela rootfs, mounts, masked paths, readonly paths e troca de root. O terceiro é o contexto de *Device Virtualization*, que modela perfis de devices, provisioning methods e políticas de acesso. O quarto é o contexto de *Security & Isolation*, que modela capabilities, user namespaces, cgroups, seccomp e LSM. Há ainda um quinto contexto de suporte, *Observability & Audit*, responsável por eventos, logs e diagnósticos.

Esses contextos não são caixas arbitrárias; eles reduzem acoplamento. Por exemplo, a decisão “terminal interativo ainda não suportado” pertence à fronteira entre *Device Virtualization* e *Lifecycle*. A decisão “em user namespace não usar `mknod`” pertence à fronteira entre *Device Virtualization* e *Security & Isolation*. Já a sequência concreta de syscalls para materializar essas decisões pertence à infraestrutura C++.

8.3 Entidades, value objects e agregados

Há um conjunto pequeno de tipos de domínio que resolve boa parte da complexidade. *ContainerId*, *RootfsPath*, *NamespaceSet*, *CapabilitySet* e *TerminalMode* são bons candidatos a value objects. *DeviceId*, composto por tipo, major e minor, também é um excelente value object porque carrega identidade técnica imutável. *DeviceNode* pode ser uma entidade quando inclui caminho, ownership e estado de provisionamento. *MinimalDevProfile* é um value object importante: ele expressa o perfil mínimo de `/dev` como uma política de domínio reutilizável. E o *ContainerAggregate* é o aggregate root que reúne pedido, plano e estado.

Uma formulação útil é a seguinte: o agregado *Container* possui uma intenção de execução; essa intenção referencia um *Rootfs*, um *RuntimeProfile* e um *DeviceProfile*; o *DeviceProfile* “MinimalDev” contém os devices padronizados deste milestone; e a camada de planejamento produz um *ExecutionPlan* derivado desse agregado. Assim, quando a política evoluir — por exemplo, adicionando `/dev/full` ou um *PtyEnabledProfile* — a mudança acontece na linguagem do domínio antes de atingir o executor.

8.4 Serviços de domínio e invariantes

Dois serviços de domínio seriam especialmente úteis. O primeiro, *MinimalDevPolicyService*, recebe contexto operacional e devolve o perfil de `/dev` aplicável. Ele decide, por exemplo, que `terminal=true` é inválido enquanto não houver `devpts`. O segundo, *ExecutionPlanningService*, traduz o agregado para um plano ordenado que a infraestrutura consegue executar.

Mais importante do que os nomes são as invariantes. Algumas delas deveriam existir como regras centrais do modelo: se `TerminalMode == Interactive`, então um *PtyProfile* completo é obrigatório; se o ambiente usa `user namespace` e o *ProvisioningMethod == Mknod*, o plano é inválido; se um device faz parte do *DevicePlan*, uma regra correspondente deve existir no *CgroupPlan*; se o caminho alvo do device não é seguro dentro do *rootfs*, o plano não pode seguir para execução. Colocar essas invariantes no domínio evita uma classe de erros em que a infraestrutura “descobre tarde” o que já era inválido desde o início.

8.5 Eventos de domínio e linguagem ubíqua

Um runtime desse tipo se beneficia bastante de eventos de domínio. Eventos como *ContainerCreationRequested*, *MinimalDevProfileSelected*, *ContainerLaunchRejected*, *DevTmpfsMounted*, *DeviceProvisioned*, *RootSwitched* e *ContainerExecFailed* não são meramente telemetria; eles ajudam a cristalizar a linguagem ubíqua do time. Em vez de falar “o helper quebrou depois de montar umas coisas”, o time passa a falar “falhou após *DeviceProvisioned*(`/dev/urandom`) e antes de *RootSwitched*”. Em infraestrutura crítica, vocabulário claro reduz custo operacional.

A linguagem ubíqua sugerida por este relatório inclui termos como “perfil mínimo de `/dev`”, “container não interativo”, “estratégia rootful por `mknod`”, “estratégia rootless por `bind mount`”, “plano de execução”, “devices `cgroup` coherence” e “TTY não suportado

neste milestone”. Esses termos devem aparecer em código, logs, métricas e documentação. Quando isso acontece, arquitetura e operação ficam alinhadas.

8.6 Cython como anti-corruption layer

Do ponto de vista de DDD, o papel mais elegante para Cython é o de anti-corruption layer. O domínio em Python fala em agregados, perfis e invariantes. O mundo do kernel fala em file descriptors, flags, `errno`, majors/minors e capabilities. Se essa fronteira for atravessada sem uma camada de tradução explícita, o domínio acaba poluído por technicalidades e a infraestrutura passa a carregar conceitos semânticos demais. Cython pode preservar essa fronteira: no lado Python, expõe métodos e exceções que façam sentido ao domínio; no lado C++, entrega structs e enums simples, estáveis e orientados à execução.

9. Padrões de Design (GoF) aplicáveis

9.1 Builder para o *Execution Plan*

O padrão *Builder* é um encaixe natural aqui porque o plano de execução é composto de várias partes que precisam ser montadas de maneira incremental e coerente. Um *ExecutionPlanBuilder* em Python pode começar com os dados brutos da requisição, incorporar defaults do runtime, aplicar o perfil mínimo de `/dev`, expandir mounts implícitos, gerar regras de cgroup e devolver um plano imutável. O Builder é útil porque evita construtores gigantes e reduz o risco de planos parcialmente válidos.

9.2 Strategy para provisionamento de devices e backend de cgroups

O padrão *Strategy* aparece em dois pontos evidentes. O primeiro é o provisionamento dos device nodes: *MknodStrategy* para rootful, *BindMountDeviceStrategy* para user namespace/rootless. O segundo é o backend do controlador de devices: uma estratégia para cgroup v1 e outra para cgroup v2/BPF [15] [16]. Essa separação evita ramificações intermináveis e faz o código refletir a realidade operacional: existem estratégias mutuamente exclusivas para atingir o mesmo resultado lógico.

9.3 State para o ciclo de vida do container

O padrão *State* ajuda o control plane a impedir transições inválidas. Um container em estado *Failed* não aceita os mesmos comandos de um container em *Running*. Um container em *Planned* ainda não tem pid/pidfd. Um container em *Creating* pode coletar eventos de setup, mas ainda não aceita operações de `exec`. Isso vale especialmente para a feature de `/dev` porque falhas de setup precisam deixar o agregado em um estado claramente diferente de um container que falhou após iniciar o processo principal.

9.4 Template Method para a pipeline de criação

A pipeline de criação de container tem uma forma muito estável: validar, preparar rootfs, montar filesystems, provisionar devices, aplicar isolamento, trocar root, dropar privilégios, executar processo. O padrão *Template Method* permite codificar essa forma em uma classe base, deixando pontos de variação para estratégias específicas de rootful/

rootless, backend de cgroup ou suporte futuro a PTY. O benefício é preservar a ordem das etapas, que em runtimes é quase tão importante quanto as etapas em si.

9.5 Abstract Factory para recursos dependentes do ambiente

Uma *Abstract Factory* pode ser usada para fornecer famílias coerentes de objetos dependentes do ambiente do host. Um exemplo: um *KernelIntegrationFactory* decide, com base em probing, se o runtime usará `openat2` ou fallback por `openat`, `clone3` ou `clone`, `pidfd` ou `waitpid` clássico. Outro exemplo: uma *SecurityBackendFactory* produz a combinação correta de implementações de devices cgroup e capabilities. Isso evita espalhar checks de kernel version/capability por toda a base.

9.6 Adapter e Facade na fronteira Python/Cython/C++

A fronteira entre Python e C++ pede explicitamente um *Adapter*. O papel do Cython é adaptar tipos, ownership e tratamento de erro. Ao mesmo tempo, a API que o domínio vê deve ser uma *Facade*: algo como `runtime_bridge.launch(plan)` é muito superior a uma coleção difusa de chamadas de baixo nível. O Adapter resolve diferenças técnicas; a Facade oferece simplicidade para o lado consumidor.

9.7 Command para passos materiais do data plane

O padrão *Command* é útil quando se deseja registrar ou replayar ações materiais do executor. Comandos como *MountTmpfsDevCommand*, *CreateDeviceNodeCommand*, *BindDeviceCommand*, *ApplyDevicesCgroupCommand* e *SwitchRootCommand* tornam o pipeline auditável e, em alguns cenários, testável por simulação. A utilidade não está em transformar tudo em objetos por formalismo; está em obter rastreabilidade de ações críticas.

9.8 Observer para telemetria e auditoria

Por fim, o padrão *Observer* é extremamente apropriado para eventos de runtime. Cada passo de setup pode publicar eventos para sinks de log, métricas e auditoria sem acoplar diretamente o executor à forma de armazenamento desses dados. Como a feature de `/dev` mínimo cruza uma região muito sensível do runtime, esse desacoplamento entre execução e observabilidade vale bastante.

10. Requisitos funcionais

RF-01. O runtime deve montar um novo `tmpfs` em `/dev` dentro do mount namespace do container, de forma que o conteúdo preexistente de `/dev` na imagem deixe de ser a superfície visível ao processo. Esse requisito existe tanto por compatibilidade quanto por segurança [2] [1] [17].

RF-02. O runtime deve provisionar exatamente os device nodes do perfil mínimo deste milestone: `/dev/null`, `/dev/zero`, `/dev/random`, `/dev/urandom` e `/dev/tty`, usando tipo, major/minor e permissões coerentes com o Linux convencional [4] [5] [6] [7].

RF-03. O runtime deve verificar, após o provisionamento de cada device, que o inode final corresponde ao tipo esperado e ao `rdev` planejado. A ausência dessa verificação abre

espaço para TOCTOU e ataques por symlink swap, especialmente em rootfs potencialmente maliciosos [26] [23] [24].

RF-04. O runtime deve suportar, em modo rootful, a criação direta dos nodes via `mknodat` quando o processo de setup possuir as capacidades necessárias, em especial `CAP_MKNOD` e privilégios adequados para mounts e root switching [13] [10] [11].

RF-05. O runtime deve suportar, em modo user namespace/rootless, uma estratégia alternativa de bind mount de device nodes confiáveis do host, ou então rejeitar explicitamente esse modo para o milestone atual. O que não pode ocorrer é uma tentativa silenciosa de `mknod` que falha tarde em runtime [14] [22] [26].

RF-06. O runtime deve derivar as regras do controlador de devices do cgroup a partir do mesmo *DevicePlan* usado para materializar os nodes no filesystem, garantindo coerência entre o que é visível e o que é acessível [15] [16] [1].

RF-07. O runtime deve rejeitar pedidos de terminal interativo, pseudo-TTY, `/dev/console`, `/dev/ptmx` ou quaisquer recursos dependentes de `devpts` enquanto o milestone atual não os implementar. A mensagem de erro deve explicar que o perfil atual é não interativo, e não apenas retornar um erro genérico de baixo nível [8] [3] [19].

RF-08. O runtime deve registrar no metadata store qual perfil de devices foi aplicado ao container e, quando houver fallback de estratégia, se o provisionamento ocorreu por `mknod` ou por bind mount. Esse dado precisa aparecer por API e por inspeção operacional.

RF-09. O runtime deve limpar corretamente recursos temporários quando o setup falhar: namespace child, cgroup intermediário, diretórios auxiliares, pipes de sincronização, locks e quaisquer descritores não reutilizados. O mount namespace ajuda nessa limpeza, mas não a substitui completamente.

RF-10. O runtime deve permitir que a ausência de controlling terminal seja distinguida da ausência de `/dev/tty`. Em outras palavras, o node precisa existir, mas o comportamento de abertura deve seguir a semântica do kernel para processos sem controlling terminal [7].

RF-11. O runtime deve detectar em startup do daemon as capacidades relevantes do host e expor esse capability matrix internamente: disponibilidade de `openat2`, `clone3`, `pidfd`, backend de cgroup e suporte prático ao modo user namespace/rootless. Isso reduz surpresas em tempo de criação e melhora a previsibilidade operacional [24] [27] [28] [16].

RF-12. O runtime deve preservar uma trilha de auditoria por etapa material do setup, incluindo montagem de `/dev`, criação de cada node, programação do cgroup de devices, troca do root e `exec` final. Essa trilha é funcionalmente relevante porque sem ela a investigação de falhas fica impraticável em produção.

RF-13. Se o runtime optar por criar symlinks como `/dev/fd` e equivalentes, isso deve ocorrer somente quando a presença de `/proc` fizer esse contrato sentido. Symlink quebrado por default piora a previsibilidade do ambiente em vez de melhorá-la [2] [26].

11. Requisitos não funcionais

RNF-01 — Segurança. Toda mutação de paths sob o rootfs deve ser resistente a symlink traversal e race conditions. A recomendação é operar por directory file descriptors, usar `O_NOFOLLOW`, preferir `openat2` quando disponível e verificar o inode final antes de aplicar mudanças sensíveis [24] [26] [23].

RNF-02 — Correção semântica. Os device nodes devem ter exatamente o tipo e o major/minor corretos, e o comportamento observado por aplicações deve coincidir com o esperado no Linux padrão. O runtime não deve inventar “substitutos funcionais” para devices do kernel quando a semântica real já existe [5] [6] [7].

RNF-03 — Princípio do menor privilégio. Capabilities como `CAP_MKNOD`, `CAP_SYS_ADMIN` e `CAP_SYS_CHROOT` devem existir apenas pelo tempo estritamente necessário ao setup, sendo removidas antes do `execve` do processo do container [13] [10] [11] [12].

RNF-04 — Observabilidade. Cada passo crítico deve gerar logs estruturados com contexto suficiente: container id, stage, path lógico, syscall, flags relevantes, errno e decisão de fallback. Métricas de sucesso/falha por etapa e tempo gasto por etapa também são recomendáveis.

RNF-05 — Portabilidade entre kernels Linux. O runtime deve degradar graciosamente quando recursos modernos não estiverem disponíveis. A ausência de `openat2` ou `clone3` não deve impedir a funcionalidade básica, desde que existam fallbacks seguros e bem testados.

RNF-06 — Desempenho. O setup de `/dev` mínimo deve adicionar overhead pequeno e previsível ao tempo de start do container. Como o conjunto de devices é minúsculo, a meta razoável é que o custo seja dominado por criação de namespace, mounts e exec, não por processamento no control plane.

RNF-07 — Manutenibilidade. O código deve preservar a separação entre política de domínio e execução privilegiada. Regras como “sem TTY neste milestone” ou “rootless usa bind mount” não devem ficar escondidas em branches profundos do C++; elas pertencem ao domínio em Python.

RNF-08 — Testabilidade. O executor C++ deve ser organizado em torno de interfaces ou facades de syscall que permitam testes de unidade, enquanto os testes de integração validam comportamento real em kernel. Um runtime sem estratégia de teste para syscalls sensíveis tende a regredir em silêncio.

RNF-09 — Governança de recursos. O uso de `tmpfs` para `/dev` deve ter tamanho e semântica de memória claros, reconhecendo que shared memory/file cache associados a `tmpfs` entram na contabilidade de memória do sistema/cgroup [17] [16].

RNF-10 — Compatibilidade declarativa. O produto deve declarar com precisão o nível de compatibilidade oferecido. “Suporta containers não interativos com perfil mínimo de `/dev`” é uma afirmação correta; “suporta TTY de forma equivalente ao Docker” não é, enquanto `devpts` não existir.

RNF-11 — Auditabilidade. O sistema deve ser capaz de explicar, após o fato, como cada device foi criado, quais regras de cgroup foram aplicadas e em qual etapa ocorreu a

falha. Isso é especialmente importante em incidentes de segurança ou investigações de compatibilidade.

12. Segurança, hardening e riscos de implementação

12.1 Não confiar no `/dev` da imagem

A primeira regra de segurança é a mais simples: o runtime não deve confiar no conteúdo de `/dev` vindo da imagem do container. Mesmo quando a imagem parece “bem comportada”, ela pode conter symlinks inesperados, device nodes indevidos, ownership estranho ou simplesmente um layout incompatível com o perfil desejado. Montar um novo `tmpfs` sobre `/dev` neutraliza essa superfície, reduz o estado herdado e torna o conjunto de devices uma whitelist explícita [2] [1] [17].

Esse princípio vale também para a própria existência do diretório alvo. O `rootfs` pode apresentar `/dev` como symlink ou como algo que não é diretório. A implementação deve tratar esse caso como potencialmente malicioso e usar resolução segura por file descriptors e/ou rejeição explícita do `rootfs`. A pior alternativa possível é concatenar strings de path e deixar o kernel seguir links arbitrários.

12.2 Symlink attacks e resolução segura de paths

Os problemas corrigidos pelo `runc` em 2025 mostram que caminhos aparentemente banais sob `/dev` podem virar vetores de ataque quando `bind mounts` ou `masked paths` são aplicados sobre alvos errados [23]. O aprendizado prático é que um runtime moderno deve operar com file descriptors o máximo possível. Abrir o `rootfs` por FD, caminhar até diretórios filhos com `openat/openat2`, criar arquivos com `O_NOFOLLOW` e usar `procd` ou APIs fd-based quando possível reduz drasticamente a superfície de TOCTOU [24] [26].

Quando `openat2` estiver disponível, vale a pena usar flags como `RESOLVE_NO_SYMLINKS` para garantir que nenhum componente do caminho dependa de links simbólicos inesperados [24]. Em kernels onde essa API não existir, o fallback precisa ser explícito e mais conservador, por exemplo restringindo a criação a diretórios previamente abertos e verificados. Em runtime, fallback não pode significar abandono de hardening.

12.3 Superfície de capabilities

`Mounts`, `pivot_root` e `chroot` exigem privilégios associados a capabilities como `CAP_SYS_ADMIN` e `CAP_SYS_CHROOT`; `mknod` depende de `CAP_MKNOD` [10] [11] [12] [13]. Em um runtime bem desenhado, essas capabilities não ficam “ativas por conveniência”. Elas são temporárias, ligadas apenas ao setup, e retiradas antes da execução do processo do usuário. Isso vale ainda mais se o executor C++ rodar em processo separado: o helper privilegiado monta e configura; o processo final do container nasce com o mínimo necessário.

12.4 Rootless/user namespaces e seu impacto real

User namespaces mudam profundamente a estratégia de segurança. Eles permitem que processos tenham capabilities relativas ao namespace, mas a documentação do kernel

deixa claro que criar devices é uma operação cujo efeito não é namespaced e, por isso, só é permitida a quem tem privilégios no initial user namespace [14]. A documentação do Docker reforça a consequência prática: `mknod` em container user-namespaced falha com permissão negada [22]. Isso significa que o runtime deve ter um caminho rootless consciente, baseado em bind mounts de devices confiáveis, e não um “best effort” que falha em tempo de execução.

12.5 Coerência entre filesystem e cgroup

Um risco menos óbvio é a incoerência entre os devices visíveis no filesystem e as permissões efetivas dadas pelo cgroup. Se o runtime cria o node mas não libera o acesso correspondente, o container falha de formas difíceis de diagnosticar. Se libera um padrão amplo demais no cgroup, pode expor acesso indevido a devices que nem sequer aparecem no `/dev` do container. Essa coerência precisa ser tratada como propriedade de segurança, não apenas de usabilidade [15] [16].

12.6 Isolamento de processo e blast radius

Outro ponto importante é o isolamento do próprio executor. Mesmo que todo o control plane esteja escrito em Python, o código que faz syscalls privilegiadas não precisa, e idealmente não deve, ficar residente no mesmo processo de longa duração da API. Um helper C++ curto, com interface pequena e logs fortes, é mais fácil de auditar, reiniciar, seccompar e observar. Isso também reduz o impacto de bugs de memória ou uso incorreto de FDs no caminho privilegiado.

12.7 Riscos de produto que parecem técnicos, mas não são

Há ainda riscos de produto que se manifestam como bugs técnicos. O principal é prometer interatividade cedo demais. Se a CLI, a API ou a documentação sugerirem suporte a TTY porque existe `/dev/tty`, usuários inevitavelmente tentarão comandos e aplicações que dependem de `devpts`. O defeito não será só técnico; será um defeito de contrato. Por isso, o hardening do runtime inclui também hardening de linguagem: recursos não implementados devem ser recusados com precisão.

13. Plano de implementação por fases

A implementação recomendada é incremental. O ganho desse desenho é permitir validar compatibilidade, segurança e arquitetura em camadas, sem tentar entregar terminal completo junto com a infraestrutura básica de `/dev`.

Fase	Objetivo	Entregável principal	Critério de aceite
Fase 0	Contrato e capability probing	ExecutionPlan, MinimalDevProfile, detecção de kernel features	API rejeita terminal interativo e planeja devices de forma determinística.
Fase 1	Rootful minimal / dev	tmpfs em /dev + mknod verificado + cgroup devices	Containers não interativos conseguem usar /dev/null, /dev/

			zero, /dev/random, /dev/urandom e /dev/tty.
Fase 2	Rootless/usersns	Bind mount strategy para devices confiáveis	Mesmo perfil mínimo funciona sem mknod em user namespaces, ou a limitação fica explicitamente recusada.
Fase 3	Hardening e observabilidade	openat2/fallback seguro, auditoria por etapa, pidfd	Testes negativos de symlink e race passam; diagnóstico operacional fica completo.
Fase 4	Compatibilidade adicional	Symlinks /dev/fd, reOpenDevNull, refinamentos de ownership	Compatibilidade com shells e utilitários melhora sem regressão de segurança.
Fase 5	Milestone TTY real	devpts, /dev/ptmx, console, PTY lifecycle	O runtime passa a suportar modo interativo com semântica semelhante ao Docker.

13.1 Fase 0 — contrato e capability matrix

Antes de tocar em syscalls, o time deve fechar o contrato do *ExecutionPlan* e o vocabulário do domínio. É aqui que nasce o *MinimalDevProfile*, o enum de *TerminalMode*, a representação do *ProvisioningMethod* e a detecção de capacidades do host. Essa fase parece burocrática, mas evita retrabalho pesado depois. Também é o momento certo para definir como Python, Cython e C++ versionam esse contrato.

13.2 Fase 1 — rootful minimal /dev

Aqui entra a primeira entrega funcional: mount namespace, tmpfs em /dev, criação dos cinco nodes por mknodat, verificação de inode, programação do cgroup de devices, troca do root e execução do processo. O foco desta fase é fechar o caminho feliz rootful e obter uma base estável de testes de integração. A meta não é terminal, nem rootless, nem refinamento de consola. A meta é um ambiente não interativo correto.

13.3 Fase 2 — suporte rootless/usersns

Se o produto quer suportar rootless, a segunda fase precisa introduzir a estratégia alternativa de bind mount. O planner passa a escolher a estratégia com base no contexto. A vantagem de tê-la separado da fase rootful é que a semântica da feature já estará estável; o time estará adicionando apenas uma nova forma de materializar o mesmo *DevicePlan*.

13.4 Fase 3 — hardening e telemetria

Nessa fase entram os refinamentos que transformam um protótipo sólido em um runtime confiável: uso de openat2 quando possível, validações adicionais por file descriptor, logs estruturados, eventos auditáveis, pidfds e cobertura de testes negativos. Também é aqui que devem entrar testes inspirados nas vulnerabilidades recentes do runc, para garantir que symlink swaps e targets inesperados não passem despercebidos [23].

13.5 Fase 4 — compatibilidade auxiliar

Com a base segura pronta, o runtime pode adicionar symlinks como `/dev/fd` e equivalentes, reabertura de `/dev/null` após troca de root quando necessário, e eventuais refinamentos de ownership e integração com `/proc`. Essa fase não muda a natureza do milestone, mas melhora a ergonomia e a compatibilidade do ambiente.

13.6 Fase 5 — devpts e modo interativo

O último passo é o grande divisor de águas. Implementar devpts significa montar `/dev/pts`, tratar `/dev/ptmx` como symlink ou bind mount correto para `/dev/pts/ptmx`, alocar PTYs, bind-montar `/dev/console` quando houver terminal, propagar resize de janela e fechar o lifecycle do PTY [3] [8] [1] [2]. Ao chegar aqui, o runtime deixa de ser apenas compatível com workloads não interativos e passa a oferecer uma experiência de terminal de verdade.

14. Estratégia de testes e critérios de aceite

14.1 Testes de unidade no domínio

No lado Python, os testes de unidade devem validar o planejamento, não o kernel. Isso inclui garantir que pedidos com `terminal=true` sejam rejeitados enquanto o milestone atual estiver ativo, que o *MinimalDevProfile* contenha exatamente os cinco devices previstos, que rootful e usersn selecionem estratégias distintas e que o *DevicePlan* e o *CgroupPlan* permaneçam coerentes. Também é valioso testar serialização do *ExecutionPlan* para a bridge Cython, porque muito bug de integração nasce de enums, flags ou paths mal traduzidos.

14.2 Testes de unidade no executor C++

No lado C++, os testes unitários devem focar em composição de syscalls e tratamento de erro. Um *SyscallFacade* mockável permite verificar se o executor chama `mount`, `mknodat`, `fstat`, `fchmod`, `pivot_root` e demais operações na ordem correta, com os flags corretos, inclusive nos caminhos de fallback e erro. O objetivo aqui não é provar que o kernel funciona; é provar que a sua lógica de coordenação funciona.

14.3 Testes de integração essenciais

Os testes de integração precisam exercitar o comportamento real observado dentro do container. Um conjunto mínimo de cenários seria suficiente para dar confiança. Em um container não interativo, escrever em `/dev/null` deve funcionar e leituras devem retornar EOF. Leituras de `/dev/zero` devem produzir bytes zero. Leitura de 16 ou 32 bytes de `/dev/urandom` deve retornar dados plausíveis sem travar indevidamente. Leitura de `/dev/random` deve seguir a semântica do kernel. O arquivo `/dev/tty` deve existir e ter major/minor corretos, mas a abertura em processo sem controlling terminal pode falhar; o teste deve reconhecer isso como comportamento válido, não como ausência de suporte [5] [6] [7].

Além do comportamento observável, os testes de integração devem verificar metadados do filesystem com `stat`: tipo do inode, `st_rdev`, permissões e ownership. Também é importante checar que o `/dev` visível ao container é de fato um mount separado, e não um diretório herdado da imagem.

14.4 Testes negativos e de segurança

Para esta feature, testes negativos valem tanto quanto os positivos. O runtime deve ser testado com imagens cujo `/dev` seja symlink para fora do rootfs, com arquivos preexistentes nos paths dos devices, com symlink swaps durante o setup e com pedidos explícitos de terminal interativo. Ele também deve ser testado em user namespace para confirmar que o caminho `mknod` não é tomado indevidamente. Esses testes não são exagero; eles refletem falhas reais já enfrentadas por runtimes maduros [23] [22].

14.5 Testes de regressão operacional

Uma vez que o runtime tiver telemetria e estados de lifecycle, vale a pena adicionar testes de regressão operacional: verificação de logs estruturados por etapa, assert de limpeza de cgroup e mounts após falha, consistência do metadata store e capacidade de inspecionar o container falho. Em sistemas de runtime, muitas regressões não quebram o “happy path”; elas quebram a capacidade de entender falhas quando o happy path não acontece.

14.6 Critérios objetivos de aceite

Uma feature como esta pode ser considerada pronta quando quatro condições forem simultaneamente verdadeiras. Primeiro, workloads não interativos conseguem usar os cinco devices previstos com semântica correta. Segundo, pedidos interativos são recusados de forma explícita e consistente. Terceiro, os devices cgroup correspondem ao que foi montado em `/dev`. Quarto, o runtime resiste a testes negativos de symlink/path traversal e mantém trilha de auditoria suficiente para explicar falhas. Sem essas quatro condições, a funcionalidade pode estar “aparentemente funcionando”, mas ainda não está madura.

15. Conclusão

A montagem de um `/dev` mínimo é uma feature pequena em aparência e estrutural em impacto. Ela toca compatibilidade Linux, isolamento, filesystem design, cgroups, capabilities, user namespaces, telemetria, modelagem de domínio e contrato de produto. Por isso, a melhor forma de tratá-la não é como uma lista de arquivos a criar, mas como um milestone completo do runtime.

A conclusão central deste relatório é que o caminho mais sólido para o seu projeto é assumir conscientemente um perfil “não interativo, porém compatível” como primeira entrega. Esse perfil deve montar um novo `tmpfs` em `/dev`, criar ou bind-mountar apenas `/dev/null`, `/dev/zero`, `/dev/random`, `/dev/urandom` e `/dev/tty`, alinhar o cgroup de devices a essa whitelist, endurecer toda manipulação de path e rejeitar

explicitamente qualquer pedido que dependa de `devpts`. Isso já entrega valor real, reduz superfície de ataque e prepara o terreno para o milestone seguinte.

Do ponto de vista arquitetural, o melhor desenho é manter Python como dono da semântica e do estado, C++ como executor determinístico de syscalls e Cython como anti-corruption layer tipada. O melhor desenho de produto é separar claramente “device compatibility básica” de “TTY interativo”. E o melhor desenho de implementação é construir a feature em fases, fechando primeiro rootful, depois rootless, depois hardening e só então PTY completo.

Se essa disciplina for mantida, o resultado não será apenas uma feature implementada. Será a fundação correta para o resto do runtime.

16. Anexos técnicos

Anexo A — Exemplo de interface Python/Cython/C++

```
# Python
plan = execution_planner.build(container_request)
launch_result = runtime_bridge.launch(plan)

# Cython (.pyx / .pxd)
cdef extern from "runtime/launch.hpp" namespace "rt":
    cdef cppclass DeviceSpec:
        string path
        char dev_type
        unsigned int major
        unsigned int minor
        unsigned int mode
        unsigned int uid
        unsigned int gid
        int provisioning_method

    cdef cppclass ExecutionPlan:
        string rootfs
        vector[DeviceSpec] devices
        vector[string] argv
        vector[string] envp
        int terminal_mode

    cdef cppclass LaunchResult:
        int pid
        int pidfd
        string error_stage
        int error_errno

    LaunchResult launch(const ExecutionPlan&) except +

# C++
LaunchResult launch(const ExecutionPlan& plan);
```

Anexo B — Vocabulário mínimo recomendado

Os termos “MinimalDevProfile”, “NonInteractiveContainer”, “ProvisioningMethod”, “DevicesCgroupPolicy”, “ExecutionPlan”, “RootSwitchStrategy”, “TTYUnsupportedInCurrentMilestone” e “KernelFeatureProbe” são boas sementes de linguagem ubíqua para o projeto. Eles carregam política, não apenas implementação.

Anexo C — Observações específicas sobre o futuro milestone de devpts

Quando o projeto evoluir para PTY real, o relatório recomenda como prioridades: montagem dedicada de devpts, criação correta de /dev/ptmx via symlink ou bind mount para /dev/pts/ptmx, bind mount de console somente quando terminal estiver ativo, integração com resize de janela e testes específicos com passwd, screen e tmux [3] [8] [19] [21]. O objetivo do próximo milestone não deve ser “ter um TTY que funciona às vezes”, mas “ter um subsistema PTY coerente”.

17. Referências

1. [1] Open Container Initiative. “config-linux.md” (runtime-spec, branch main). GitHub. <https://github.com/opencontainers/runtime-spec/blob/main/config-linux.md>. Acesso em 12 mar. 2026.
2. [2] opencontainers/runc. “libcontainer/SPEC.md” (branch main). GitHub. <https://github.com/opencontainers/runc/blob/main/libcontainer/SPEC.md>. Acesso em 12 mar. 2026.
3. [3] Linux Kernel Documentation. “devpts”. <https://docs.kernel.org/filesystems/devpts.html>. Acesso em 12 mar. 2026.
4. [4] Linux Kernel Documentation. “devices.txt”. <https://www.kernel.org/doc/Documentation/admin-guide/devices.txt>. Acesso em 12 mar. 2026.
5. [5] Michael Kerrisk (man-pages). “null(4)”. man7.org. <https://man7.org/linux/man-pages/man4/null.4.html>. Acesso em 12 mar. 2026.
6. [6] Michael Kerrisk (man-pages). “random(4)”. man7.org. <https://man7.org/linux/man-pages/man4/random.4.html>. Acesso em 12 mar. 2026.
7. [7] Michael Kerrisk (man-pages). “tty(4)”. man7.org. <https://man7.org/linux/man-pages/man4/tty.4.html>. Acesso em 12 mar. 2026.
8. [8] Michael Kerrisk (man-pages). “pts(4)”. man7.org. <https://man7.org/linux/man-pages/man4/pts.4.html>. Acesso em 12 mar. 2026.
9. [9] Michael Kerrisk (man-pages). “mount_namespaces(7)”. man7.org. https://man7.org/linux/man-pages/man7/mount_namespaces.7.html. Acesso em 12 mar. 2026.
10. [10] Michael Kerrisk (man-pages). “mount(2)”. man7.org. <https://man7.org/linux/man-pages/man2/mount.2.html>. Acesso em 12 mar. 2026.
11. [11] Michael Kerrisk (man-pages). “pivot_root(2)”. man7.org. https://man7.org/linux/man-pages/man2/pivot_root.2.html. Acesso em 12 mar. 2026.

12. **[12]** Michael Kerrisk (man-pages). “chroot(2)”. man7.org. <https://man7.org/linux/man-pages/man2/chroot.2.html>. Acesso em 12 mar. 2026.
13. **[13]** Michael Kerrisk (man-pages). “capabilities(7)”. man7.org. <https://man7.org/linux/man-pages/man7/capabilities.7.html>. Acesso em 12 mar. 2026.
14. **[14]** Michael Kerrisk (man-pages). “user_namespaces(7)”. man7.org. https://man7.org/linux/man-pages/man7/user_namespaces.7.html. Acesso em 12 mar. 2026.
15. **[15]** Linux Kernel Documentation. “Device Whitelist Controller” (cgroup v1). <https://docs.kernel.org/admin-guide/cgroup-v1/devices.html>. Acesso em 12 mar. 2026.
16. **[16]** Linux Kernel Documentation. “Control Group v2”. <https://docs.kernel.org/admin-guide/cgroup-v2.html>. Acesso em 12 mar. 2026.
17. **[17]** Michael Kerrisk (man-pages). “tmpfs(5)”. man7.org. <https://man7.org/linux/man-pages/man5/tmpfs.5.html>. Acesso em 12 mar. 2026.
18. **[18]** Docker Docs. “Alternative container runtimes”. <https://docs.docker.com/engine/daemon/alternative-runtimes/>. Acesso em 12 mar. 2026.
19. **[19]** Docker Docs. “docker container run”. <https://docs.docker.com/reference/cli/docker/container/run/>. Acesso em 12 mar. 2026.
20. **[20]** Docker Blog. “Docker 0.9: Introducing Execution Drivers and libcontainer”. <https://www.docker.com/blog/docker-0-9-introducing-execution-drivers-and-libcontainer/>. Acesso em 12 mar. 2026.
21. **[21]** Docker Docs. “Prior releases”. <https://docs.docker.com/engine/release-notes/prior-releases/>. Acesso em 12 mar. 2026.
22. **[22]** Docker Docs. “Isolate containers with a user namespace”. <https://docs.docker.com/engine/security/users-remap/>. Acesso em 12 mar. 2026.
23. **[23]** opencontainers/runc. “CHANGELOG.md” (branch main). GitHub. <https://github.com/opencontainers/runc/blob/main/CHANGELOG.md>. Acesso em 12 mar. 2026.
24. **[24]** Michael Kerrisk (man-pages). “openat2(2)”. man7.org. <https://man7.org/linux/man-pages/man2/openat2.2.html>. Acesso em 12 mar. 2026.
25. **[25]** Michael Kerrisk (man-pages). “getrandom(2)”. man7.org. <https://man7.org/linux/man-pages/man2/getrandom.2.html>. Acesso em 12 mar. 2026.
26. **[26]** opencontainers/runc. “libcontainer/rootfs_linux.go” (branch main). GitHub. https://raw.githubusercontent.com/opencontainers/runc/main/libcontainer/rootfs_linux.go. Acesso em 12 mar. 2026.
27. **[27]** Michael Kerrisk (man-pages). “clone(2)”. man7.org. <https://man7.org/linux/man-pages/man2/clone.2.html>. Acesso em 12 mar. 2026.
28. **[28]** Michael Kerrisk (man-pages). “pidfd_send_signal(2)”. man7.org. https://man7.org/linux/man-pages/man2/pidfd_send_signal.2.html. Acesso em 12 mar. 2026.